

Game room recommendation sorting

Difficulty: Hard

Expect Time: 60min

Author: Aren

The objective is to implement a factory function that produces a custom comparator suitable for `std::sort`. This comparator must order a collection of `RoomPtr` objects based on a hierarchical set of priority rules. The implementation must strictly adhere to the requirements of strict weak ordering to ensure predictable and stable behavior during the sorting process.

Definitions (`game_room_sort.h`)

The following structures and interfaces are provided to facilitate the implementation:

```
enum class RoomType { Casual, Ranked, Coop, Tournament, Puzzle };
```

Interface `IRoom` Provides access to room metadata:

```
int GetId() const;
RoomType GetType() const;
int GetPlayerCount() const;
```

Type Aliases:

```
using RoomPtr = std::shared_ptr<IRoom>;
using RoomCompare = std::function<bool(const RoomPtr&, const RoomPtr&)>;
```

You Implement the factory function:

```
RoomCompare BuildRoomCompare(const std::vector<int>& idPriority,
                             const std::vector<RoomType>& typePriority);
```

This factory must return a callable object (e.g., a lambda or `std::function`) that defines the "less-than" relationship between two `RoomPtr` instances. The comparator must impose a **Strict weak ordering** (compatible with `std::sort`).

Strict weak ordering

- A comparator `cmp(a, b)` represents the relation " $a < b$ " and must satisfy:
 - Irreflexive: `cmp(x, x)` is always `false` for any `x`.
 - Asymmetric: if `cmp(a, b)` is `true`; then `cmp(b, a)` must be `false`.
 - Transitive: if `cmp(a, b)` and `cmp(b, c)` are `true`, then `cmp(a, c)` must be `true`.
 - Equivalence transitivity: if neither `cmp(a, b)` nor `cmp(b, a)` holds, `a` and `b` are considered equivalent; this equivalence must be transitive.

Hierarchical Sorting Rules

When comparing two rooms, you must apply these three rules in order. If they match on one rule, move to the next:

1. **ID priority:** rooms whose `id` appears in `idPriority` come first. Their relative order is the same as the index order in `idPriority`.
2. **Type priority:** among rooms not selected by ID, those whose `type` appears in `typePriority` come next. Their order is determined by the order in `typePriority`.
3. Remaining rooms come last.

Tie-Breaking (same priority)

If two rooms end up in the same priority group, follow these steps to decide which one comes first:

1. **Player Count:** Sort by the number of players in descending order (more players = higher priority).
2. **ID Ascending:** If the player counts are also equal, the room with the smaller ID comes first. This ensures the sort is "deterministic," meaning you get the exact same result every time you run it.

Hint

To ensure the comparator is both efficient and robust, you must adhere to the following technical requirements when implementing `BuildRoomCompare`.

- The comparator must **ONLY COMPARE** two `RoomPtr`s — it must **NOT** perform sorting or **mutate** global state.
- To implement efficiently, precompute lookup maps for ID ranks and Type ranks so the overall sort remains $O(n \log n)$.

The resulting comparator must be a lightweight, stateless callable (e.g., a **lambda** capturing the precomputed lookup maps by value or reference). It must only perform comparison logic and must not trigger further sorting operations or modify any global states.

Edge Case

- Empty Priority Lists: The logic should correctly default to the "Remaining Rooms" category if the priority lists are empty.
- Unknown Types: Handle `RoomType` values that are not present in the `typePriority` list.
- Duplicate IDs: If an ID appears multiple times in `idPriority`, adhere to the first occurrence's index for rank determination.
- Empty Input: The comparator should safely handle scenarios where the provided list of rooms is empty.rules.

Input

1. Complete the required class implementation. The following files already contain some implementations:
 - `game_room_sort.h`
2. The Online Judge system will replace the following files. **DON'T** write implementation in those file:
 - `entrypoint.cpp`
 - `test.h`
 - `case.h`
3. The test cases for student available in `case*.h` .

Output

Online Judge will run the test:

1. **DON' T** output any information to the terminal.
2. When any test failed, Online Judge will try to list error messages.